

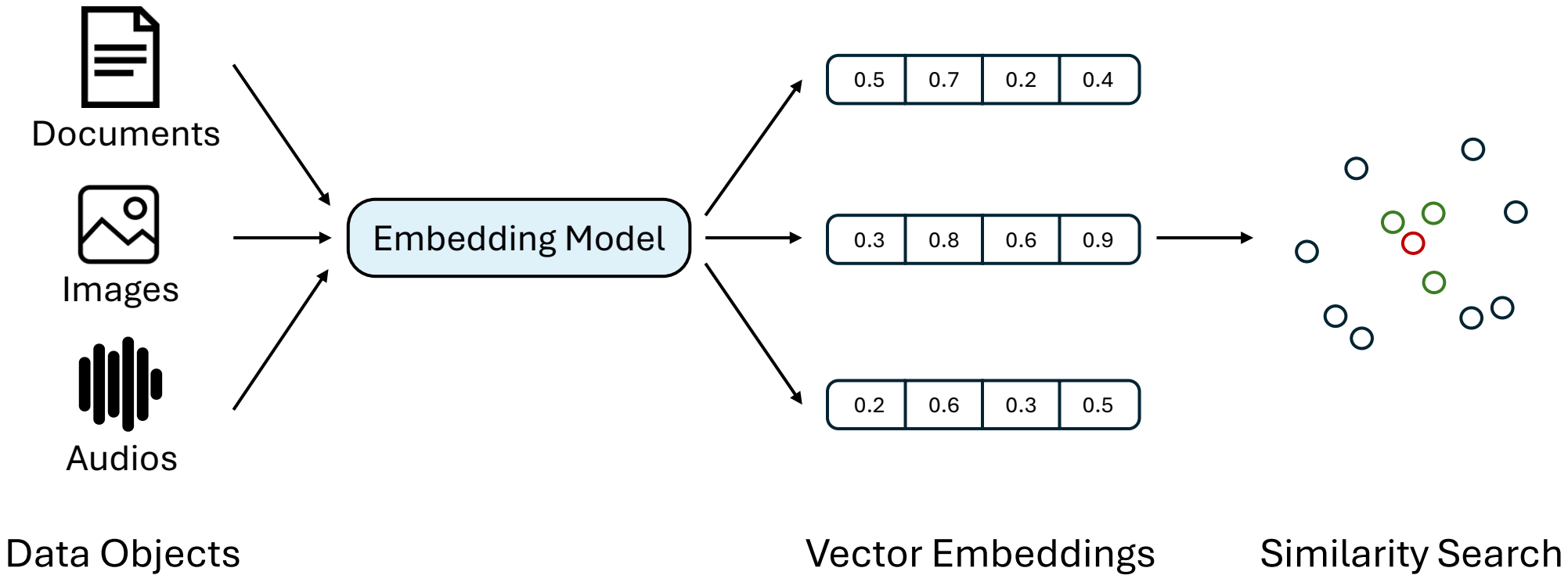
DiskJoin: **Large-scale** Vector Similarity Join **with SSD**

Yanqi Chen, Xiao Yan, Alexandra Meliou, Eric Lo

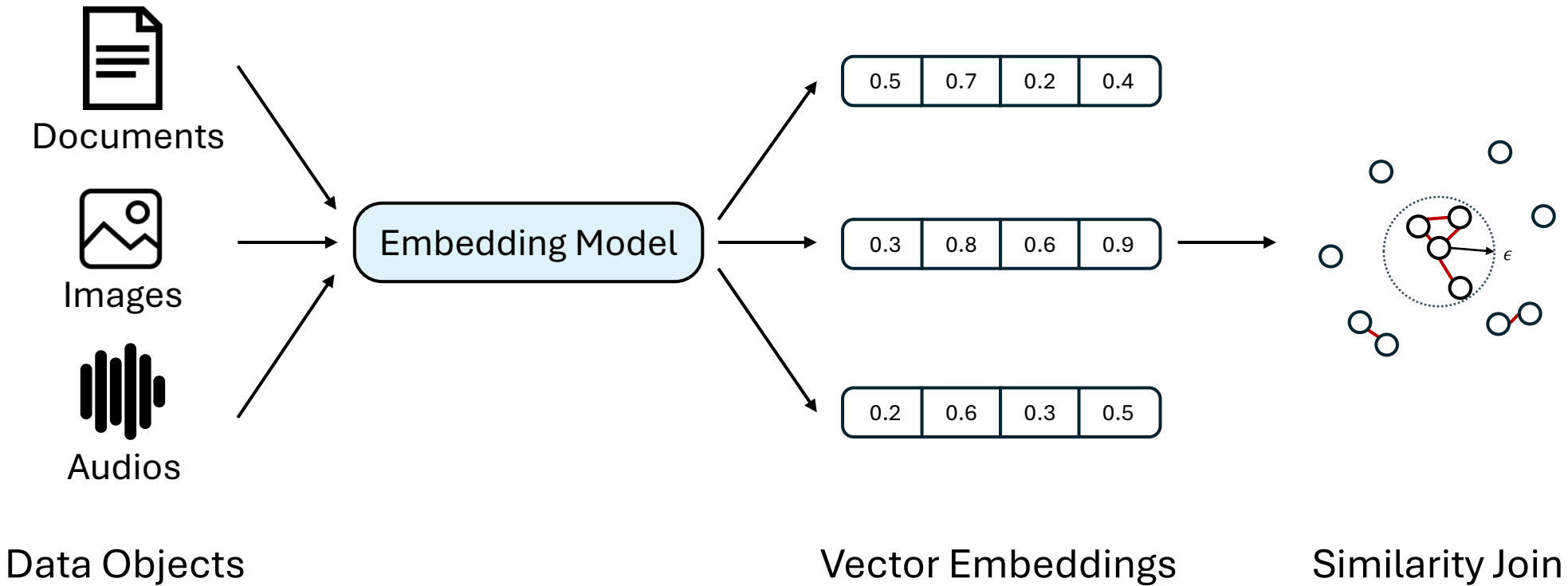
University of Massachusetts Amherst

1/16/2026

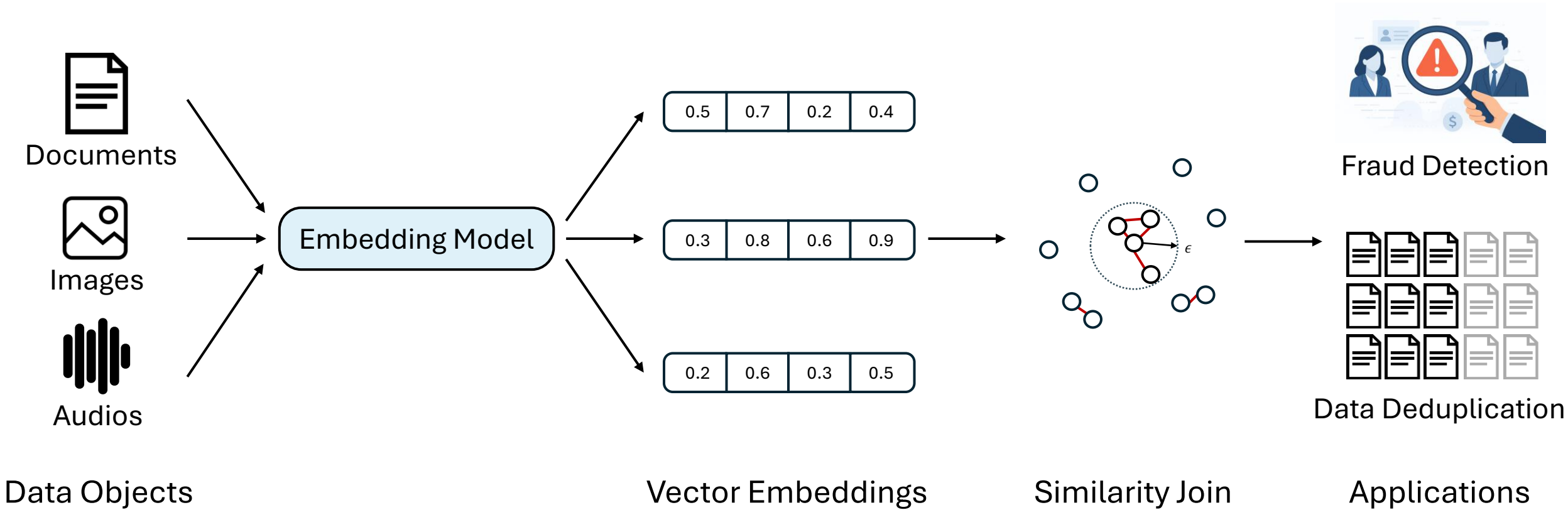
Vector Embeddings and Similarity Join



Vector Embeddings and Similarity Join



Vector Embeddings and Similarity Join



Scaling to Large Data Sizes

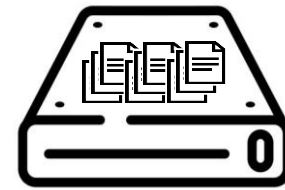


I have an **1TB** text dataset.

I want to deduplicate it using **similarity join**.

But it cannot fit in RAM, how should I complete the task?

RAM



Local Disk

Scaling out: Distributed Methods

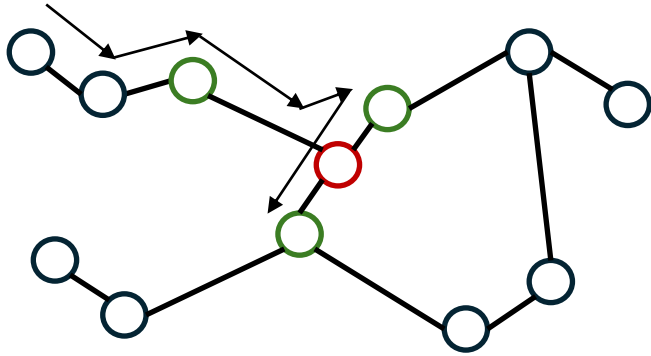
- Prior work has explored distributed methods
 - e.g., MAPSS, ClusterJoin, C2Net
 - Distribute dataset to multiple machines
 - Conduct computation via MapReduce

X Require tedious cluster setup

X High cost: CPU, RAM, disk for each machine

X Inefficient: high inter-machine communication cost

Similarity Join with SSD: Index Choice

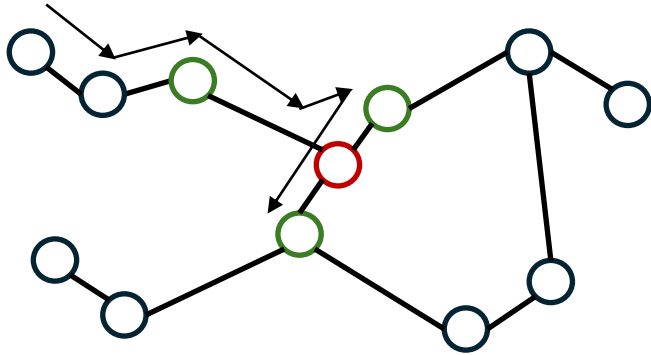


Graph Index

DiskANN

[Subramanya et al., NeurIPS 2019]

Similarity Join with SSD: Index Choice

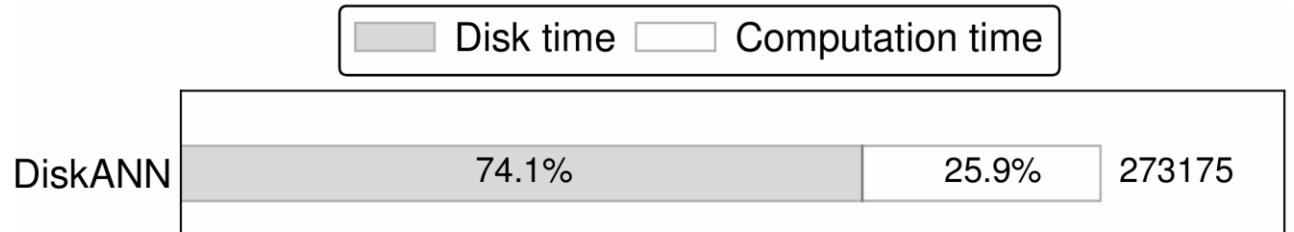


Graph Index

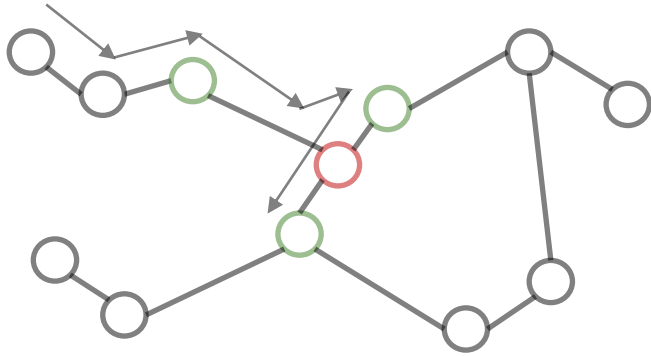
DiskANN

[Subramanya et al., NeurIPS 2019]

Read amplification!



Similarity Join with SSD: Index Choice

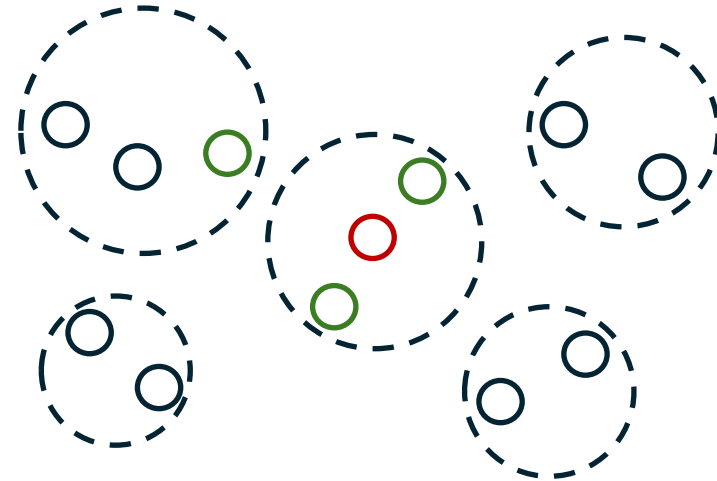


Graph Index

DiskANN

[Subramanya et al., NeurIPS 2019]

Read amplification!



Cluster Index

Read block of vectors ✓

Similarity Join with SSD: Index Choice



	Total (GB)	Useful (GB)	Amp.
DiskJoin	181	180.3	1.0039
DiskANN	139,485	19,427	7.18

Graph Index

Cluster Index

DiskANN

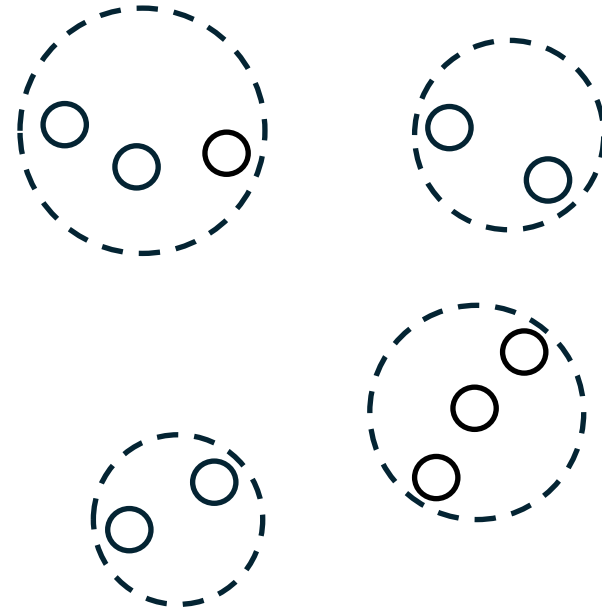
[Subramanya et al., NeurIPS 2019]

Read block of vectors ✓

Read amplification!

Cluster-based Framework

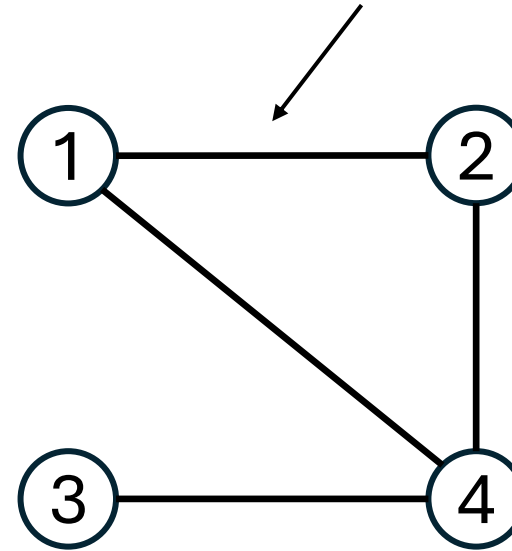
1. Cluster the vectors
2. Identify candidate cluster pairs
3. For each candidate pair
 1. Load clusters from disk
 2. Compare vectors across clusters



Cluster-based Framework

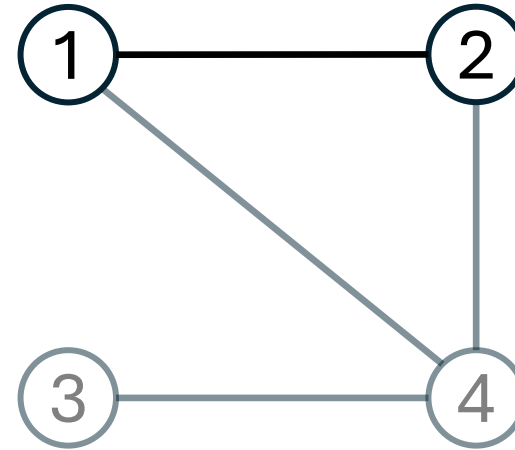
1. Cluster the vectors
2. Identify candidate cluster pairs
3. For each candidate pair
 1. Load clusters from disk
 2. Compare vectors across clusters

Cluster pair that may contain joinable vectors

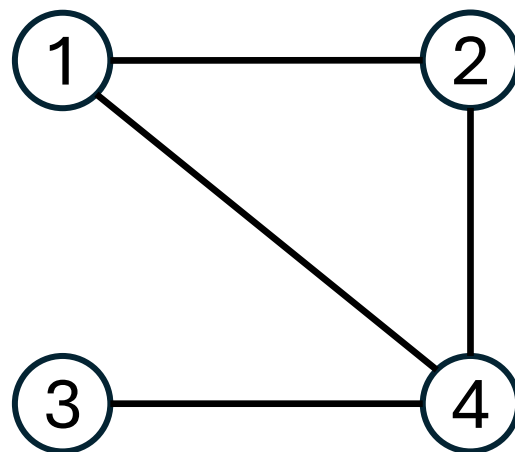


Cluster-based Framework

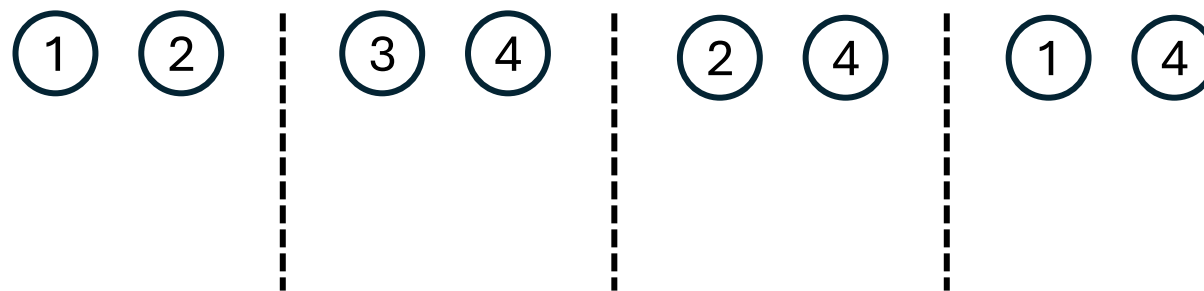
1. Cluster the vectors
2. Identify candidate cluster pairs
3. For each candidate pair
 1. Load clusters from disk
 2. Compare vectors across clusters



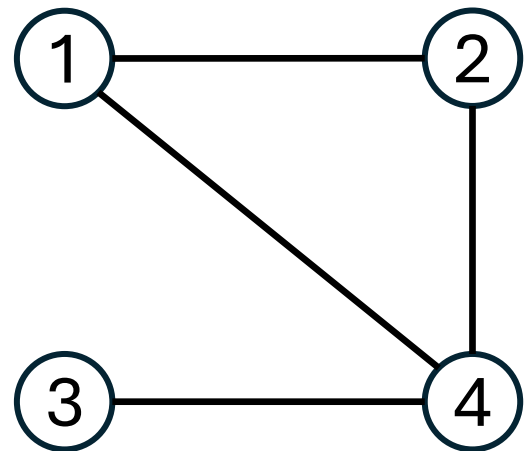
Repetitive Data Access



Candidate pairs



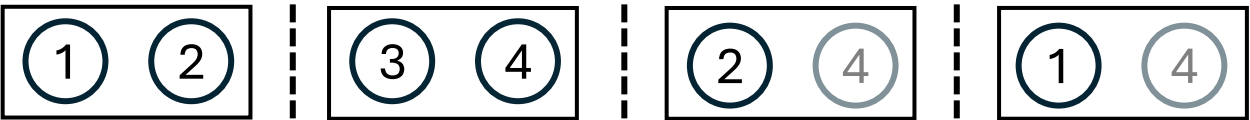
Data Reuse via Caching



Candidate pairs

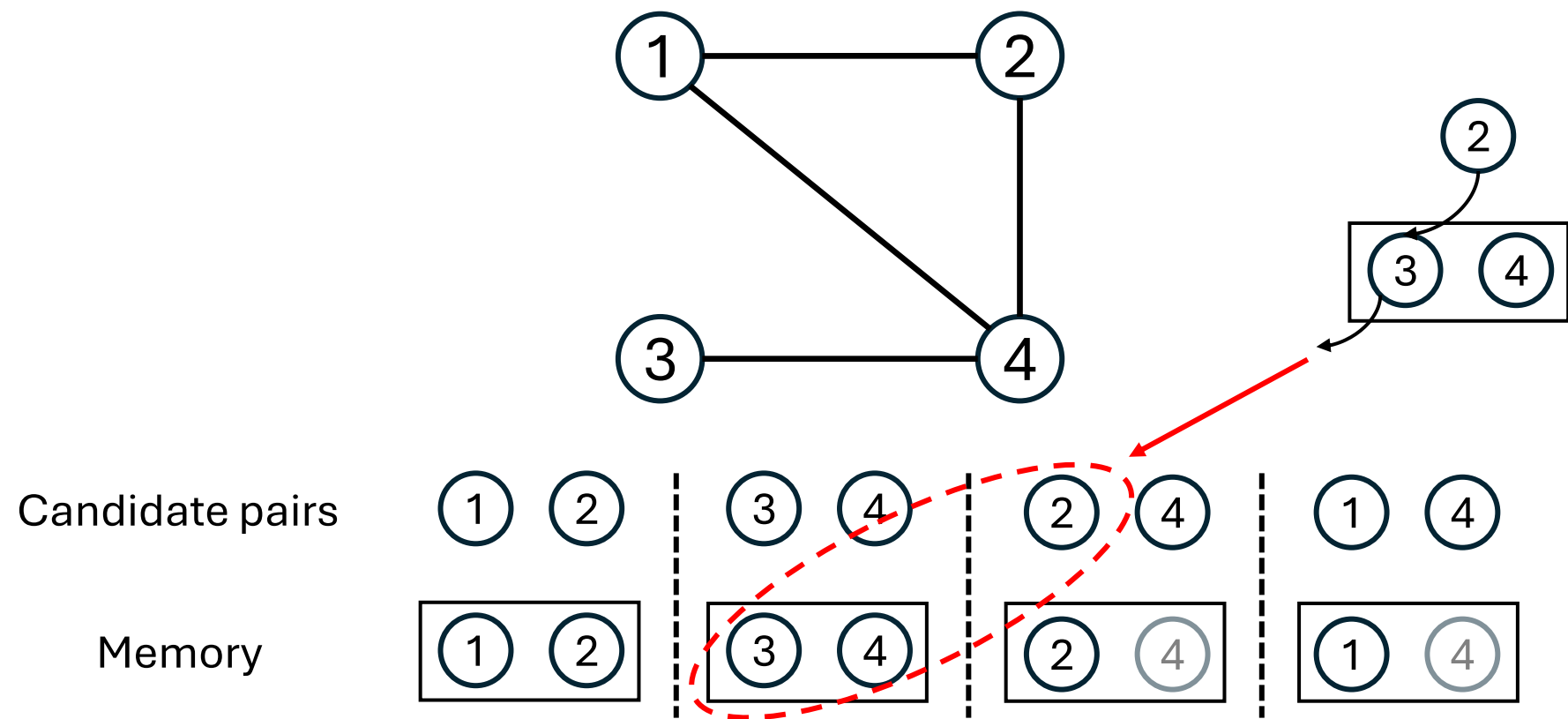


Memory



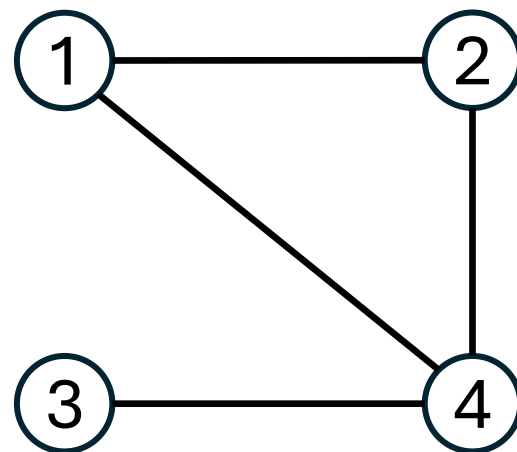
Disk read time -> Cache miss rate

Data Reuse via Caching

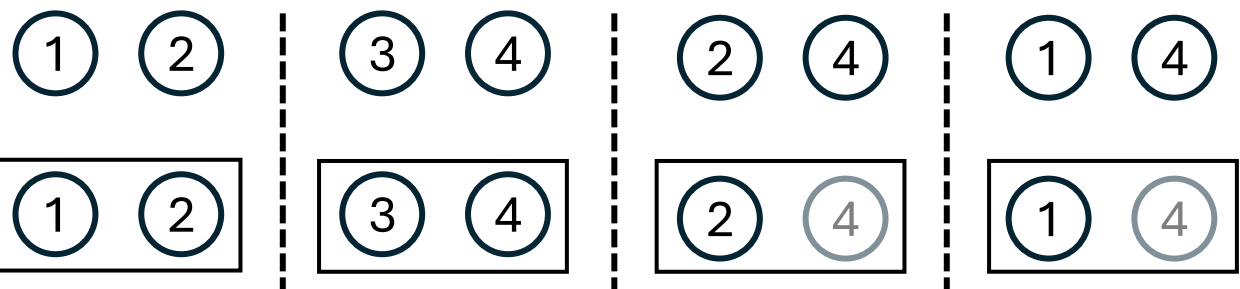


Which cluster to evict matters

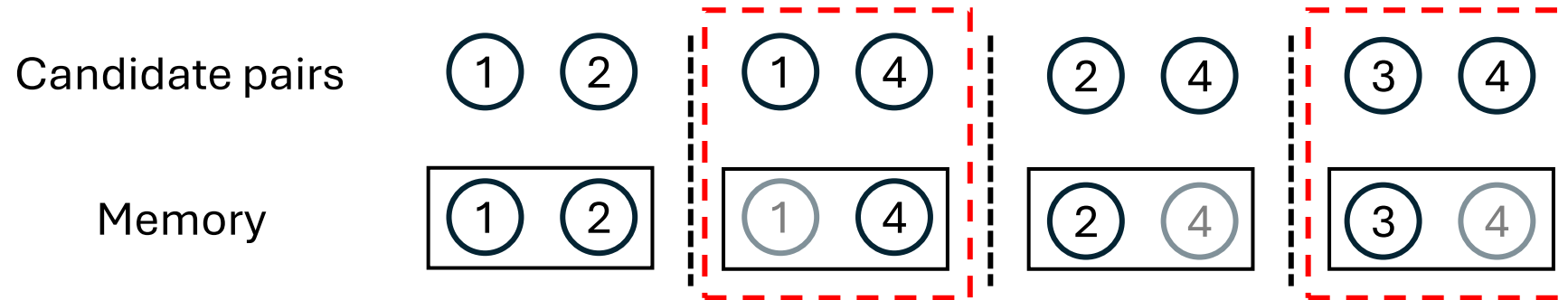
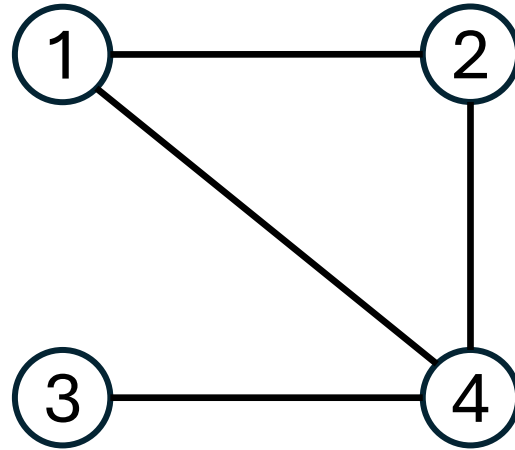
Data Reuse via Caching



Candidate pairs

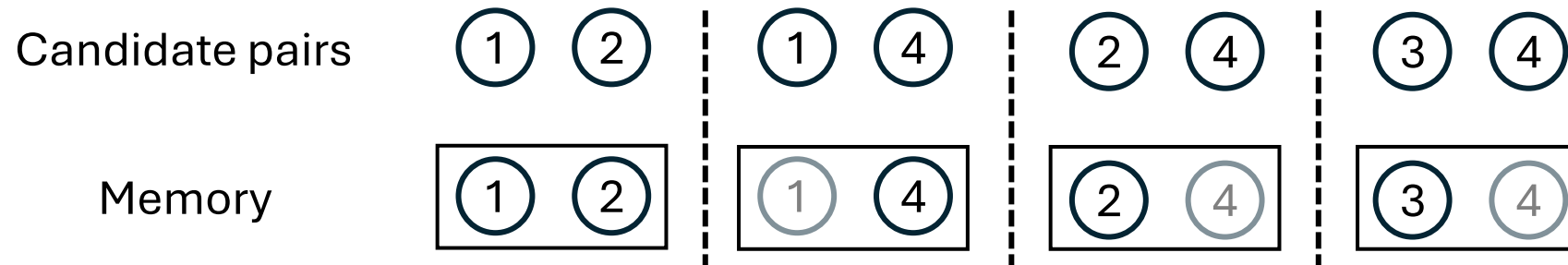
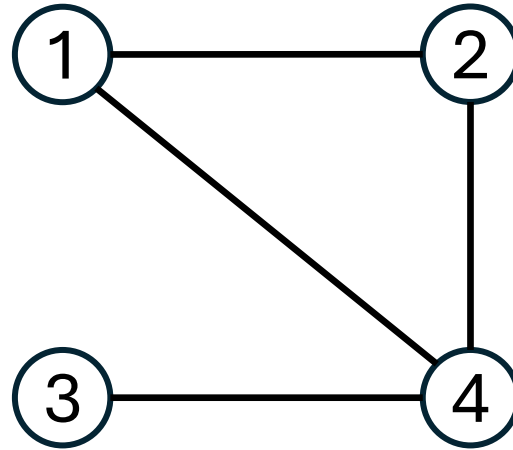


Data Reuse via Caching



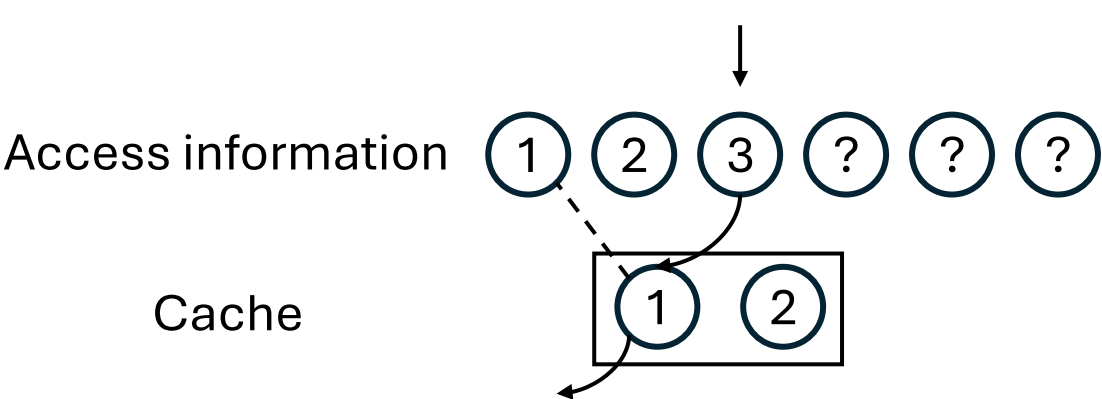
Processing order also matters

Data Reuse via Caching



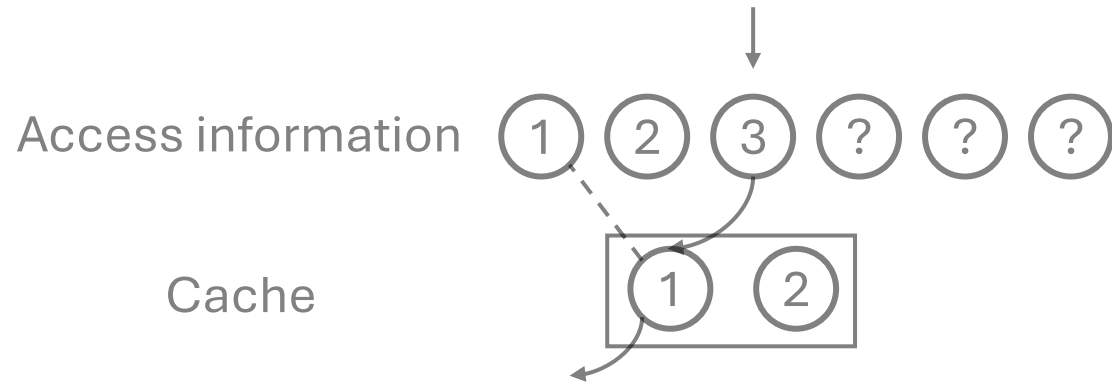
How to decide the optimal **eviction policy** and **processing order**?

Optimal Eviction Policy

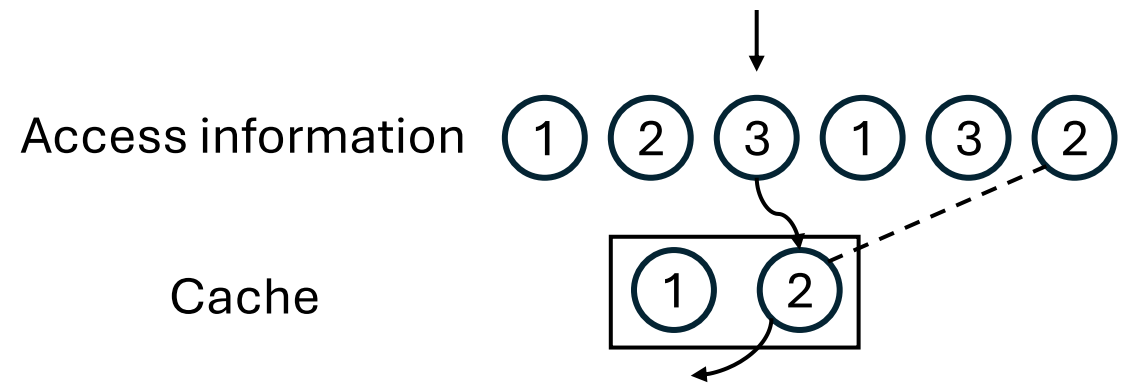


LRU: looks in past

Optimal Eviction Policy



LRU: looks in past

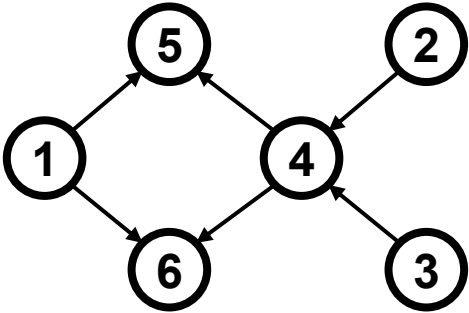


Belady's: looks in **future**

Provably optimal!

Heuristic for Ordering

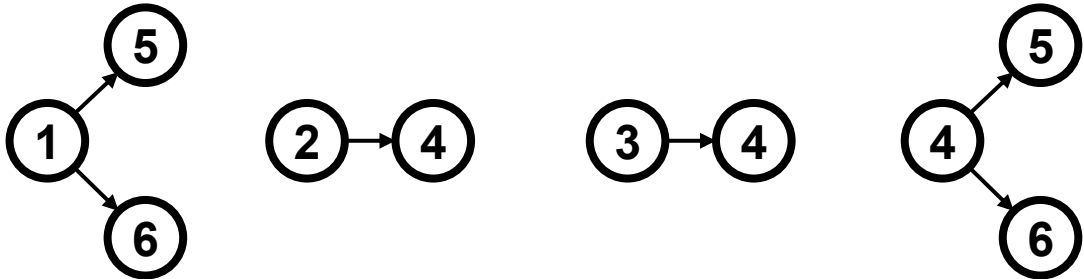
Intuition 1: Group by node



dependency graph



$2|E|$

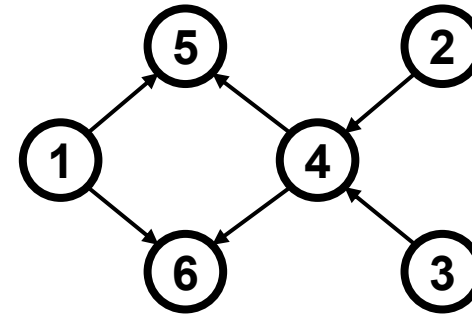


$|V|+|E|$

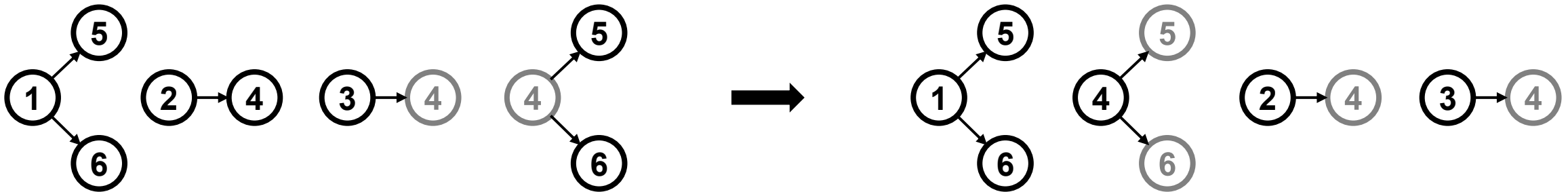
Heuristic for Ordering

Intuition 1: Group by node

Intuition 2: Place nodes with high neighbor overlap together



dependency graph

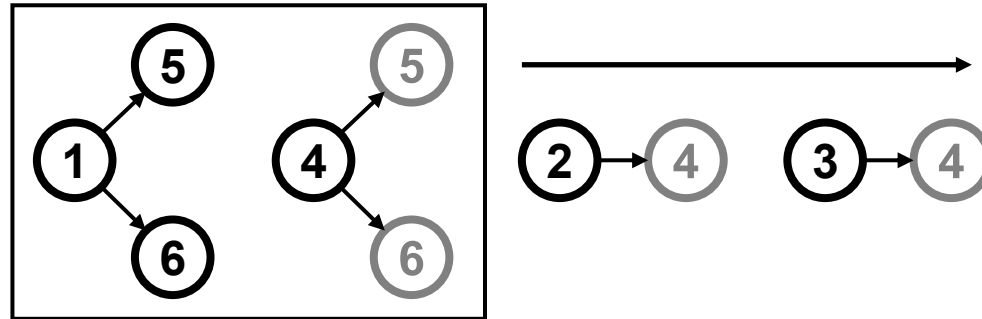


Processing Order Optimization

$$\text{maximize } F(P) = \sum_{\underbrace{0 < P(v) - P(u) \leq w}_{\text{Sliding window}}} \underbrace{|N(u) \cap N(v)|}_{\text{Neighbor overlap}}$$

$$w = 2$$

$$F(P) = 2 + 1 + 1 = 4$$

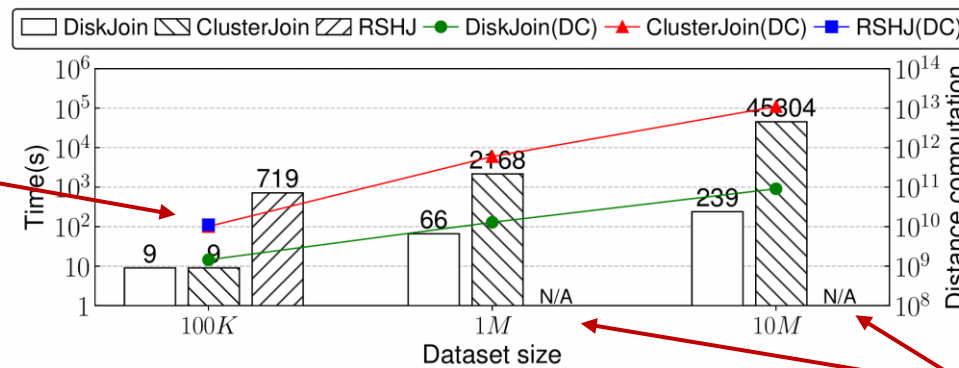


Greedy [Wei et al., SIGMOD 2016]: $P[i] = \underset{v}{argmax} \sum_{j=i-w}^{i-1} |N(P[j]) \cap N(v)|$

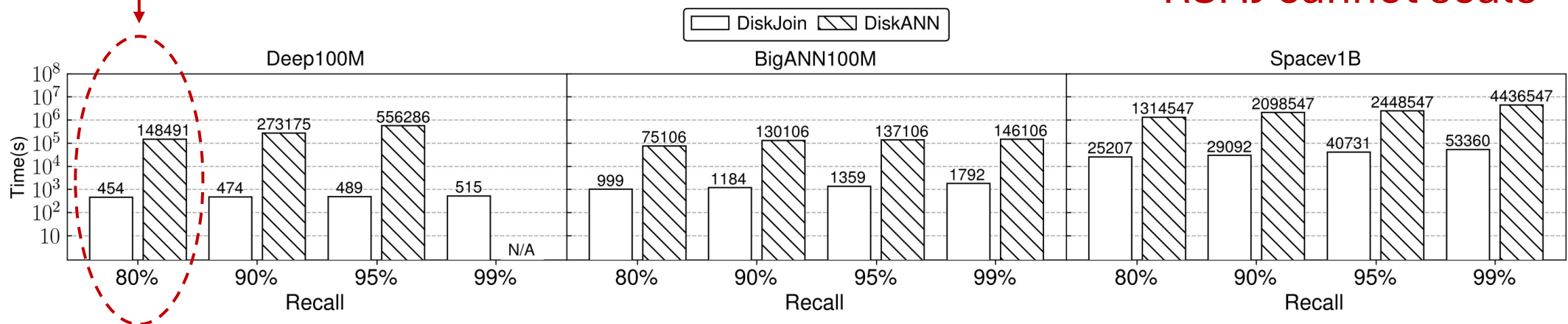
Experiments

- DiskJoin outperforms distributed methods and DiskANN by orders of magnitude

DiskJoin is 2-3 orders of magnitude faster

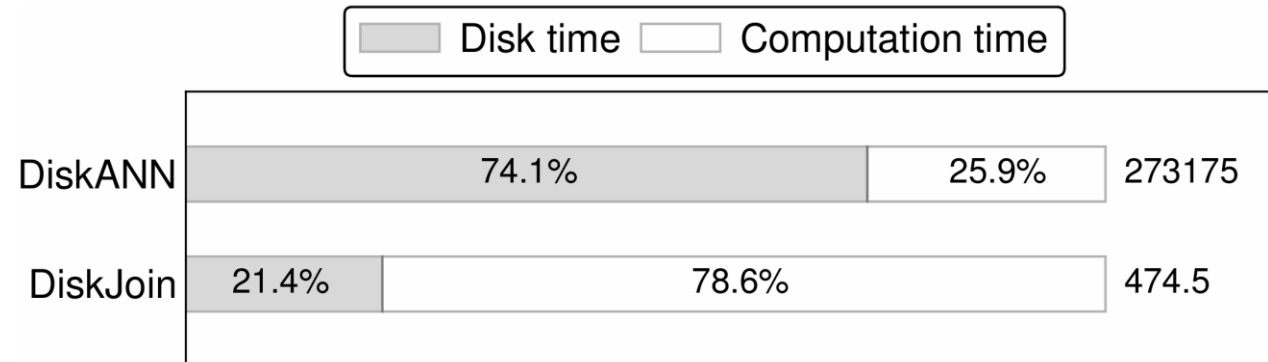


RSHJ cannot scale



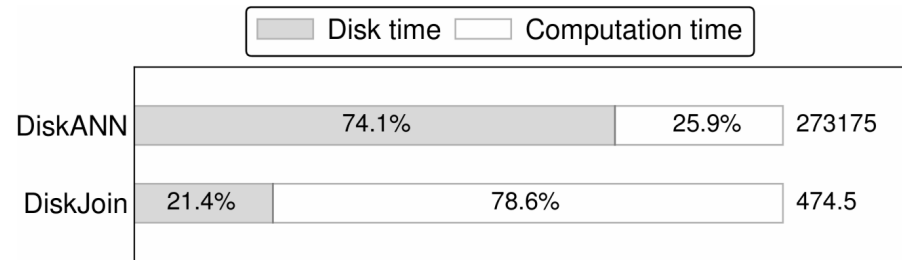
Experiments

- DiskJoin successfully eliminates the disk access time bottleneck

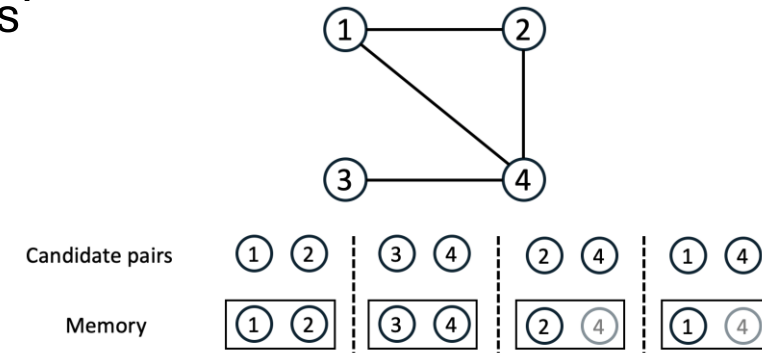


Summary

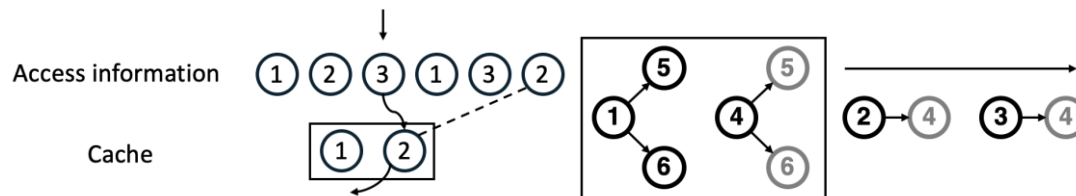
- Identify the disk bottleneck of disk similarity join due to read amplification and repetitive data access.



- Based on cluster-based framework, introduce memory caching to achieve data reuse.



- Minimize cache miss rate by optimal cache management with Belady's algorithm and task ordering.



- Evaluation results show DiskJoin outperforms competitors by orders of magnitude.

